TÉCNICO LISBOA

Distributed and Parallel Computing

MPI Report — 2D Grid

Pedro Rio
MSc Data Science and Engineering
Instituto Superior Técnico
97241

Sebastião Maia Serqueira
MSc Data Science and Engineering
Instituto Superior Técnico
97108

Eduardo Machado
MSc Data Science and Engineering
Instituto Superior Técnico
97133

1 Program Overview

The problem is a matrix-factorisation recommender: a sparse user–item matrix A is approximated by $A \approx L \cdot R$, with L of size $users \times features$ and R of size $features \times items$. The factors are fitted by gradient descent over the observed entries, and the final dense product $B = L \cdot R$ yields one recommendation per user. The four stages are *ReadInput*, *InitialLR*, *UpdateLR* (repeated for the requested number of iterations), and *FilterFinalMatrix*.

2 Why a 2D Grid

The simplest MPI design is *replicate-all*: every process keeps full copies of L and R, splits the non-zero list into blocks, computes a partial gradient, and sums it with a global `MPI_Allreduce`. It is correct and short, but it does not scale: the reduced gradient has $O(features \cdot (users + items))$ entries and that volume *does not shrink* as processes are added, while compute per process falls as $1/P$. Communication therefore dominates at scale, and on *wide* instances (a large R) it *anti-scales* — adding processes makes it slower. Memory does not scale either: a single node must hold the whole problem.

Both limits have the same cause — replication — so we distribute the factors over a 2D process grid.

3 The 2D-Grid Version

3.1 Process mesh

`MPI_Dims_create` factors P into a balanced $P_r \times P_c$ grid; `MPI_Cart_create` builds the mesh (with reorder disabled, so a rank's grid id equals its world rank) and `MPI_Cart_sub` derives a *row* and a *column* sub-communicator for each rank.

3.2 Foster methodology

Partitioning. L is split by *user-rows* across the grid rows and R by *item-columns* across the grid columns. A non-zero (u, i) is owned by rank $(userBlock(u), itemBlock(i))$, so each rank holds exactly the L-rows and R-columns its non-zeros touch — memory per rank is $O((users \cdot features + features \cdot items)/\sqrt{P})$.

Communication. Each rank computes a partial gradient from its cell's non-zeros (all data local), then dL is summed across the *row* communicator and dR down the *column* communicator. These are reductions of $\sqrt{P}$-smaller slices over $\sqrt{P}$-sized sub-communicators, not a global reduce of the whole gradient.

Agglomeration and Mapping. The non-zeros are agglomerated into one contiguous block per grid cell and routed there once with a bucketed `MPI_Scatterv`; one cell maps to one process.

3.3 Deterministic, communication-free init

`initialLR` is seeded with a fixed value, but a partitioned factor cannot simply seek into the random stream (an R-column slice is strided). The trick is *generate-and-discard*: every rank runs the same `srandom(1)` sequence in the serial fill order but stores only the values that fall in its slice. The slices hold exactly the serial values, with no broadcast of the factors.

3.4 Distributed final stage

$B = L \cdot R$, the mask of known entries, and the per-user arg-max are all computed on the cells: each rank forms its B-block, takes a local arg-max over its item columns, and the results are combined across the row with `MPI_MAXLOC` (smallest item index on ties, matching the serial scan) and gathered to the root in user order. There is no serial-root tail.

4 A Note on the Hybrid Variant

This report covers the **pure MPI** deliverable: all parallelism comes from MPI processes, each rank runs its share sequentially, and the binary links no OpenMP runtime. A second deliverable, `mpi-hybrid/`, keeps this same 2D-grid decomposition and adds OpenMP threads inside each rank; it has its own report. Every figure below was measured with the pure variant.

5 Evaluation

Table 1 compares wall-clock time of the 2D-grid version against the naive replicate-all design it replaced.

Provenance. The **grid** rows are the committed measurements in `compare/data/comparison.mpi.csv`, produced by `compare/scripts/benchmark.py` on an Apple Silicon laptop (macOS 15.6, 10 cores, 8 of them performance cores), three repetitions with the minimum kept, every run verified against the instance's `.out` file. They are reproducible from this repository by one command. The **replicate** rows were measured on the same machine before the rewrite and *cannot* be regenerated here: that implementation was replaced and no longer exists in the tree. They are retained because the comparison is the reason the 2D grid exists, and labelled so no one mistakes them for current figures.

Table 1: Replicate-all vs. 2D-grid (time, s). Grid rows: measured now, this laptop. Replicate rows: pre-rewrite, same machine, not regenerable.

instance	design	1	2	4	8
inst200 (wide)	replicate*	7.62	5.26	5.36	10.64
	grid	6.07	3.94	2.25	2.94
inst500	replicate*	8.50	4.84	3.28	8.41
	grid	8.48	4.76	2.66	3.30
inst1000	replicate*	4.15	2.70	1.89	3.03
	grid	3.96	2.22	1.23	0.99

*pre-rewrite implementation, not regenerable from this repository.

The grid version is faster at every process count on every instance, and the gap widens with P: at 8 processes it is 3.6× faster on the wide instance and 3.1× on inst1000. Replicate-all *regresses* sharply past 4 processes — 10.64 s on the wide instance, worse than its own single-process time — because the dense `Allreduce` of the whole gradient becomes a communication ceiling.

The grid version is not immune to the 8-process point either: it regresses on the wide instance (2.25 → 2.94 s) and on inst500 (2.66 → 3.30 s), while still improving on inst1000 (1.23 → 0.99 s). Two effects overlap there, and this measurement cannot separate them: the machine has 10 cores but only 8 performance cores, so an 8-process run competes with the operating system and the timing harness for the last of them; and the per-process work at that point is small enough that communication is a larger share of it. The honest claim is therefore that the grid design *degrades far more gracefully*, not that it scales indefinitely — a 2.94 s regression against 10.64 s is a difference in kind, but it is still a regression.

6 Implications for Parallel and Distributed Computing

6.1 Distribute the data, not just the work

Replicate-all distributes the *work* but not the *data*, so it hits both a memory wall and a communication ceiling. The 2D grid distributes the data: the factor slices and the gradient fall as $1/\sqrt{P}$, and the dense B built in the final stage — which actually dominates the footprint on every instance here, by roughly a factor of five — falls as $1/P$. Replicate-all's per-rank memory does not fall at all. The general lesson is that a scalable design must shrink each process's footprint,

not only its share of the compute; and that the term worth quoting is the one that dominates, not the one that reads best.

6.2 Match the collective to the decomposition

The win is not a faster `Allreduce`; it is a *smaller* one, over a *smaller* communicator. Because a user's updates are spread only across its grid row and an item's only down its grid column, the global reduction decomposes into independent row and column reductions. Choosing the communicator to match the data layout is what turns flat communication into communication that scales.

6.3 Remove the serial tail

Computing B and the arg-max on the root would be an $O(users \cdot items \cdot features)$ sequential tail that, by Amdahl's law, caps the speedup. Distributing it over the same grid keeps the whole pipeline parallel.

6.4 Reproducibility under reordering

The partitioned reductions sum the gradient in a different order than the serial code, so floating-point results differ in the last bits — but the arg-max is robust to that, so the recommendations match the serial output byte for byte at every process count and grid shape. A deterministic serial reference is what makes this verifiable, and the test suite asserts it rather than assuming it: 33 cases covering every instance at 1, 2 and 4 processes, non-square and prime grids, blocks left empty when processes outnumber users or items, and the same output when one configuration is run twice.